LIMITBEYOND

FITNESS & GYM MANAGEMENT PLATFORM

API Specification

Version 1.0 · REST contract for mobile, web console and public surface

Document ID	**LB-API-1.0**
Status	**Draft for review**
Date	**17 September 2026**
Prepared by	**Bibek Jha**
Implements	**LB-SRS-1.1 · LB-ARCH-1.0 · LB-ENG-1.0 · LB-DATA-1.0**
Base path	`/api/v1`
Machine contract	**OpenAPI 3.1, generated by springdoc**

> **What is authoritative here.** The generated OpenAPI document is the machine contract and the source of the TypeScript clients; when it and this document disagree, the OpenAPI wins and this is corrected. What this document adds is the part a schema cannot express — why a resource is shaped the way it is, which rules are enforced where, and the handful of conventions that must hold across every endpoint for the API to be safe rather than merely functional.

Table of Contents

1. Introduction and Design Stance

2. Conventions

   Versioning · naming · scoping · types · errors · pagination · idempotency · rate limiting · caching

3. Authentication and Authorisation

4. Auth Endpoints

5. Platform and Subscriptions (Super Admin)

6. Gyms and Locations

7. People — Members, Trainers, Assignment

8. Billing — Plans, Memberships, Payments

9. Attendance

10. Scheduling — Classes, Bookings, Personal Training

11. Training — Exercises, Workouts, Nutrition, Progress

12. Engagement — Feedback, Notifications

13. Insights and Reporting

14. Media

15. Public API — Directory, Timetables, Leads

16. Webhooks

17. OpenAPI, Clients and Deprecation Policy

18. Appendix A — Status Codes

19. Appendix B — Error Code Catalogue

20. Appendix C — Worked Examples

1. Introduction and Design Stance

1.1 What this API serves

One HTTP interface serves three consumers, and the differences between them shape the design more than any style preference:

Consumer	Characteristics	What that demands
Mobile app all five roles	Intermittent connectivity, old builds in circulation for weeks after a store release	Idempotent writes, small payloads, and a version that keeps working long after the next one ships (`NFR-MNT-04`)
Web console admins	Good connectivity, large lists, bulk operations, exports	Cursor pagination, filtering, and asynchronous report generation
Public site unauthenticated	Server-rendered at build and revalidation time; crawled by search engines	Cacheable reads, stable shapes, and exactly one narrow write path

1.2 Three rules that outrank style

Most API conventions are arbitrary but worth being consistent about. Three here are not arbitrary, because getting them wrong produces a security or correctness defect rather than an inelegant interface.

> **1 — A resource outside the caller's scope returns 404, never 403.** A 403 confirms that the resource exists. Applied across a multi-tenant directory, that turns identifier enumeration into a census of every gym's members. `NFR-SEC-01a` forbids exactly this, so the absence of authority and the absence of the resource must be indistinguishable to the caller. The one exception is an action on a resource the caller *can* see but may not perform — that is a genuine 403.

> **2 — Every retryable write requires an `Idempotency-Key`.** A network failure is ambiguous: the client cannot distinguish a request that never arrived from one that succeeded but whose response was lost. Without a key, retrying risks a duplicate check-in or a double-charged payment and not retrying risks losing it. With one, the client may retry freely. This is also how `FR-ATT-05`'s duplicate-rejection requirement is met under offline replay — one mechanism satisfying a correctness requirement and an availability requirement at once.

> **3 — Scope is taken from the path or the token, never from the body.** No endpoint accepts `gymId` or `locationId` as a request field. Both are derived server-side from the route and the caller's grants. A body-supplied tenant identifier is a cross-tenant write waiting to happen, and it is the specific defect the public lead endpoint would otherwise be vulnerable to (§15.3).

1.3 Deliberately absent

Not in this API	Why
HATEOAS / hypermedia links	Both clients are written against a generated, typed contract. Runtime link discovery would add payload weight and ceremony that nothing consumes.
GraphQL	Considered and declined in ADR-13: it complicates RLS context propagation and per-field authorisation for no benefit the mobile client needs.
Partial-response / field selection	Resources are already shaped to their use. Arbitrary field selection defeats response caching and makes the generated types meaningless.
Bulk write endpoints	Aside from the explicit admin bulk actions in §13, a write touches one resource. Bulk semantics ("7 of 10 succeeded") are ambiguous under retry.

Not in this API	Why
WebSockets / SSE	ADR-10: no requirement needs a server-initiated stream. Waitlist promotion is a push notification; live views poll.
Payment collection	Withdrawn from scope entirely. Money is recorded, never handled (`CON-04`).

2. Conventions

2.1 Versioning and base path

All authenticated endpoints sit under `/api/v1`; the public surface under `/api/v1/public`. The major version is in the path and increments only for a breaking change. Additive changes — a new optional field, a new endpoint, a new enum value in a response — ship within `v1` and clients must tolerate them.

> **Why path versioning rather than a header.** App-store review means an old mobile build stays in circulation for weeks after its replacement ships (`NFR-MNT-04`). A version visible in the URL is trivially routable, cacheable and greppable in logs when diagnosing which build a failing client is running. Header negotiation is more elegant and less operable.

2.2 Naming and resource shape

Rule	Example
Plural nouns, kebab-case	`/check-ins`, `/pt-packages`, `/class-types`
Never a verb in a path	The method is the verb.
State transitions are sub-resources	`POST /bookings/{id}/cancellation` — not `/cancelBooking`
Location-scoped resources nest	`/locations/{locationId}/members` — the scope is in the route, which is also what the service authorises against
Gym-wide resources do not nest	`/plans`, `/class-types` — gym comes from the token (`FR-ORG-12`)
Own data uses `/me`	`/me/workouts`, `/me/progress` — every account has these, whatever its role (`FR-PFP-01`)
JSON fields are `camelCase`	Configured once on the serialiser, never per field

> **`/me` is not a convenience alias.** It is the API's expression of the dual-facet model. `GET /me/workouts` is the same endpoint for a member, a trainer and a gym owner, and it takes no role parameter. Had personal training data been nested under `/members/{id}`, staff would have needed a member identity to use it — which is precisely the modelling mistake `FR-PFP-01` exists to prevent.

2.3 Data types on the wire

Type	Representation	Notes
Identifier	UUID string	UUIDv7. Unguessable in URLs, time-ordered for index locality.
Timestamp	ISO-8601 with offset	`"2026-09-17T06:30:00+05:30"`. Always sent with an offset; stored UTC.
Date	`YYYY-MM-DD`	For things without a time: `expiresOn`, `joinedOn`.
Time of day	`HH:mm`	Opening hours, class start times on a recurring rule.
Money	`{ "amountMinor": 250000, "currency": "INR" }`	**Integer minor units.** ₹2,500.00 is `250000` paise. No floating point appears anywhere near an amount, in any language, at any layer.
Enum	Upper snake case string	`"WAITLISTED"`. Clients must tolerate unknown values — new ones are additive.
Phone	E.164	`"+919876543210"`. Never appears in a URL path or query.
Duration	Integer, unit in the name	`durationMinutes`, `restSeconds` — never an unqualified number.

2.4 Errors — RFC 7807 Problem Details

One shape for every failure, produced by a single handler (`LB-ENG-1.0` §3.5). Clients switch on `code`; `detail` is for humans and may change without notice.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type":          "about:blank",
  "title":         "No sessions remaining",
  "status":        422,
  "detail":        "This personal training package has no sessions left.",
  "code":          "NO_CREDITS_REMAINING",   // stable — switch on this
  "correlationId": "01J9F2K7X3QW8M4N"          // quote to find it in the logs
}
```

Validation failures add a `fields` array naming the offending field and a message — **never the submitted value**, which may be personal data and would then be logged by whatever consumes the response.

```
{
  "title":  "Validation failed",
  "status": 400,
  "code":   "VALIDATION_FAILED",
  "fields": [
    { "field": "phone", "message": "must be a valid Indian mobile number" }
  ]
}
```

> **What never appears in an error response:** stack traces, SQL, class names, upstream provider messages, or the value that failed validation. Error responses are the most reliable reconnaissance surface an attacker has. The correlation ID is the bridge — the user quotes it, and it locates the full detail in the logs.

2.5 Pagination — cursor, not offset

```
GET /api/v1/locations/{locationId}/members?limit=50&cursor=eyJpZCI6IjAxSjl...

{
  "items": [ … ],
  "nextCursor": "eyJpZCI6IjAxSjlGMks3WDNRVzhNNE4ifQ",  // null on the last page
  "hasMore": true
}
```

Parameter	Behaviour
`limit`	Default 25, maximum 100. A larger value is clamped, not rejected.
`cursor`	Opaque. Clients must not parse or construct one — its encoding is free to change.

> **Why not `?page=2`.** Offset pagination re-runs the query and skips rows, so anything inserted or deleted between page one and page two shifts every subsequent row — items get duplicated across pages or skipped entirely. Attendance and bookings change constantly while an admin is paging through them, so offsets would be visibly wrong in exactly the lists people use most. Cursors anchor to a row and are stable under concurrent writes.

2.6 Filtering and sorting

```
GET /locations/{id}/members?status=ACTIVE&expiringBefore=2026-10-31&sort=-joinedOn
GET /locations/{id}/check-ins?from=2026-09-01&to=2026-09-30&purpose=SHIFT
```

Filters are explicit named parameters, never a generic query language. Each is a documented, indexed access path; an arbitrary filter expression would let a client construct a query that scans a ten-million-row table. Sorting uses `sort=field` or `sort=-field` for descending, restricted to an allow-list per endpoint.

2.7 Idempotency

Required on every `POST` a client may retry: check-ins, workout logs, bookings, payments, leads. Optional and harmless elsewhere.

```
POST /api/v1/locations/{locationId}/check-ins
Idempotency-Key: 9f1c8b30-6a4e-7c21-b8d5-1e2f3a4b5c6d
```

Situation	Response
First request	Processed normally. Key and outcome recorded.
Replay, same key, same payload	**The original response**, with `Idempotency-Replayed: true`. No second row is created.
Same key, *different* payload	`409 IDEMPOTENCY_KEY_REUSED`. Reusing a key for a different operation is a client bug and silently accepting it would hide it.
Key absent where required	`400 IDEMPOTENCY_KEY_REQUIRED`.

> **The key belongs to the operation, not the attempt.** A client that generates a fresh UUID per retry defeats the entire mechanism — each retry looks like a new operation and creates a duplicate. This is the single most consequential client-side rule in the API, and it is why the mobile outbox generates its key once at enqueue and reuses it across every attempt.

Keys are retained 24 hours, which comfortably exceeds any plausible offline queue.

2.8 Rate limiting

Every response to a rate-limited endpoint carries the standard headers, so a client can back off intelligently rather than hammering until it is blocked.

```
RateLimit-Limit: 5
RateLimit-Remaining: 2
RateLimit-Reset: 47              // seconds
Retry-After: 47                  // on 429 only
```

Endpoint group	Limit	Why
OTP request	5 / hour per number, 20 / hour per device	Each one costs real money and is the most abusable endpoint in the product.
OTP verify	10 / hour per number	Brute-forcing a six-digit code.
Sign-in (password)	10 / hour per account	Credential stuffing.
Check-in	10 / hour per user	Generous — offline replay must never trip it.
Public lead submission	3 / hour per IP, 2 / day per phone	The only unauthenticated write path in the platform (§15.3).
Authenticated default	300 / minute per user	A ceiling, not a throttle.

2.9 Caching and conditional requests

Surface	Policy
Public directory reads	`Cache-Control: public, max-age=300, stale-while-revalidate=3600` plus an `ETag`. These are served to a CDN and a static-site build; they must be cacheable.
Authenticated reads	`Cache-Control: private, no-store` by default. A shared cache must never hold one gym's member list.
Reference data exercises, muscle groups	`private, max-age=3600` with an `ETag`. The exercise catalogue is large and changes rarely — `If-None-Match` turns a repeat fetch into a 304.
Media	Never proxied. Cloudinary URLs are returned and the CDN serves the bytes (ADR-15).

2.10 Standard headers

Header	Dir.	Purpose
`Authorization`	→	`Bearer <access token>`
`Idempotency-Key`	→	UUID, one per logical operation (§2.7)
`X-Gym-Id`	→	*Optional.* Only meaningful for an account holding grants in more than one gym; selects the active gym. Always validated against the token's grants — never trusted.
`X-Request-Id`	→	Client-supplied correlation id. Echoed back and written to every log line for that request.
`X-Correlation-Id`	←	Server-assigned when the client sent none. Also appears in error bodies.
`Idempotency-Replayed`	←	`true` when a stored result was returned rather than the work being done again.
`Deprecation` / `Sunset`	←	Present on endpoints scheduled for removal (§17.3).

3. Authentication and Authorisation

3.1 Token model

Token	Lifetime	Contents and handling
Access	15 minutes	JWT carrying `sub`, account state, and the account's role grants with their location scope. Sent as a bearer token. Short-lived because it cannot be revoked before expiry.
Refresh	60 days	Opaque, stored server-side hashed, revocable individually or wholesale. Exchanged for a new pair; rotated on every use.

Clients store both in the device secure store — Keychain or Keystore, never `AsyncStorage` or `localStorage` (`LB-ENG-1.0` §7).

3.2 Grants in the token

```
{
  "sub":   "01J9F2K7X3QW8M4N5P6R7S8T9V",
  "state": "ACTIVE",
  "grants": [
    { "role": "TRAINER",        "gymId": "01J9…A1",
      "locationIds": ["01J9…L1", "01J9…L2"], "gymWide": false },
    { "role": "LOCATION_ADMIN", "gymId": "01J9…A1",
      "locationIds": ["01J9…L1"],           "gymWide": false }
  ],
  "exp": 1789012345
}
```

> **Read that example carefully — it is the whole authorisation model in one payload.** This account is a trainer at two locations and an admin at only one. At `L1` it may verify members; at `L2` it may coach but not administer. Effective authority is the **union of grants evaluated per location**, never the maximum applied everywhere (`FR-AUTH-28`). Trainer + Location Admin is a first-class supported combination (`FR-AUTH-26`), not an edge case.

3.3 How a request is authorised

#	Stage	What happens
1	Token validation	Signature and expiry. Failure → `401 UNAUTHENTICATED`.
2	Account state	`PENDING_VERIFICATION` reaches only `/me/profile` and `/me/status` (`FR-AUTH-09`). Suspended or deactivated → `403`.
3	Gym liveness	A lapsed gym past its grace period restricts administrative endpoints; members keep read access to their own data (`FR-SUB-07`).
4	Permission check	`auth.require(permission, locationId)` in the service method — scope first, permission second.
5	RLS context	`SET LOCAL` injects gym, locations and role into the transaction. Every query is filtered by the database regardless of what the code asked for.

Stages 4 and 5 are deliberately redundant. A mistake in either is caught by the other; a breach requires both to be wrong at once.

4. Auth Endpoints

Method	Path	Auth	Purpose
POST	`/auth/otp/request`	None	Send a one-time password by SMS or email. Rate-limited hard (§2.8).
POST	`/auth/otp/verify`	None	Exchange a code for a token pair. Creates the account on first successful verification.
POST	`/auth/password/sign-in`	None	Email and password alternative (`FR-AUTH-02`).
POST	`/auth/password/reset-request`	None	Always returns 202, whether or not the address exists — see below.
POST	`/auth/password/reset`	Token	Complete a reset with the emailed token.
POST	`/auth/refresh`	Refresh	Rotate the token pair.
POST	`/auth/sign-out`	Bearer	Revoke the presented refresh token.
POST	`/auth/sign-out/all`	Bearer	Revoke every session (`FR-AUTH-13`).
GET	`/auth/sessions`	Bearer	List active sessions with device labels.
POST	`/auth/register`	Bearer	Complete registration against a gym and one location. Account enters `PENDING_VERIFICATION` .

> **Two endpoints deliberately refuse to be informative.** `/auth/otp/request` returns 202 whether or not the number is registered, and `/auth/password/reset-request` returns 202 whether or not the address exists. Distinguishing them would turn either endpoint into an account-existence oracle — a way to discover which phone numbers belong to gym members. The unhelpfulness is the feature.

REQUEST AND RESPONSE

```
POST /api/v1/auth/otp/request
{ "identifier": "+919876543210", "channel": "SMS" }
→ 202 Accepted
{ "expiresInSeconds": 300, "resendAfterSeconds": 30 }

POST /api/v1/auth/otp/verify
{ "identifier": "+919876543210", "code": "481920" }
→ 200 OK
{
  "accessToken":  "eyJhbGciOi…",
  "refreshToken": "rt_9f1c8b30…",
  "expiresIn":    900,
  "user": {
    "id": "01J9…", "firstName": "Anjali", "lastName": "Rao",
    "state": "ACTIVE",
    "grants": [ { "role": "MEMBER", "gymId": "01J9…A1",
                  "locationIds": ["01J9…L1"], "gymWide": false } ]
  }
}
```

5. Platform and Subscriptions

Super Admin only. This is the surface by which gyms are admitted to and removed from the platform.

Method	Path	Purpose
GET	`/platform/gyms`	All gyms, filterable by `state`. Cross-tenant — the one place that is legitimate.
GET	`/platform/gyms/pending`	The verification queue.
POST	`/platform/gyms/{id}/verification`	Approve or reject. Rejection requires a reason (`FR-AUTH-05`).
POST	`/platform/gyms/{id}/subscription-payments`	**Records cash received and makes the gym live** (`FR-SUB-03`).
POST	`/platform/gyms/{id}/suspension`	Suspend for cause, with a reason.
DELETE	`/platform/gyms/{id}/suspension`	Reinstate.
GET	`/platform/subscriptions/due`	Worklist: expiring, in grace, or lapsed (`FR-SUB-11`).
GET	`/platform/subscription-offerings`	Term and price catalogue.
POST	`/platform/subscription-offerings`	Define an offering.
POST	`/platform/gyms/{id}/admins`	Super Admin override: appoint or transfer Gym Owner (`FR-AUTH-23`).
GET	`/platform/metrics`	Cross-gym counts and growth.

```
POST /api/v1/platform/gyms/{gymId}/subscription-payments
Idempotency-Key: 3b7e…
{
  "offeringId": "01J9…OF1",
  "amountMinor": 1200000,          // ₹12,000.00
  "currency":    "INR",
  "method":      "CASH",
  "reference":   "Collected in person, receipt 0042",
  "receivedOn":  "2026-09-17"
}
→ 201 Created
{ "subscriptionId": "01J9…SB1", "gymState": "LIVE",
  "startsOn": "2026-09-17", "expiresOn": "2027-09-16" }
```

6. Gyms and Locations

Method	Path	Required	Purpose
GET	`/gym`	Any member	The caller's own gym profile.
PATCH	`/gym`	Gym Owner	Profile, branding, GSTIN. Gym-wide (`FR-ORG-12`).
GET	`/locations`	Any member	Locations the caller may access.
POST	`/locations`	Gym Owner	Create. Location Admins cannot (`FR-ORG-03`).
PATCH	`/locations/{id}`	Admin at that location	Hours and contact details only, unless Gym Owner.
POST	`/locations/{id}/closure`	Gym Owner	Close a location (`FR-ORG-14`).
GET	`/locations/{id}/closures`	Any at location	Holiday calendar.
POST	`/locations/{id}/closures`	Admin at location	Declare a closure; cancels affected sessions.
GET	`/locations/{id}/qr-token`	Admin at location	Current rotating check-in payload for display at the desk.
GET	`/admins`	Gym Owner	Every admin with scope and who appointed them (`FR-AUTH-25`).
POST	`/admins`	Any admin	**Promote to Location Admin** — see below.
DELETE	`/admins/{grantId}`	Appointer or Owner	Demote. Removes only the admin grant (`FR-AUTH-21`).

```
POST /api/v1/admins
{
  "userId":      "01J9…U7",
  "locationIds": ["01J9…L1"]      // REQUIRED and non-empty — FR-AUTH-17
}
→ 201 Created

// Attempting to grant a location outside your own scope:
→ 403 { "code": "SCOPE_EXCEEDED",
        "detail": "You can only grant authority over locations you administer." }

// Attempting to demote the final admin of a location:
→ 409 { "code": "LAST_ADMIN",
        "detail": "This location would be left with no administrator." }
```

> **An empty `locationIds` is rejected, not defaulted.** A promotion that does not name a location has no safe interpretation: treating it as gym-wide would silently hand a floor manager authority over every branch. `FR-AUTH-17` requires the named scope, and the API refuses rather than guessing.

7. People — Members, Trainers, Assignment

Method	Path	Required	Purpose
GET	`/locations/{id}/members`	Admin / Trainer	Roster. Filters: `status`, `planId`, `expiringBefore`, `trainerId`, `q`.
GET	`/members/{id}`	Admin at their location	Full record. **404 if outside scope**, never 403.
PATCH	`/members/{id}`	Admin at location	Administrative fields only — never the member's own profile fields.
GET	`/locations/{id}/pending-verifications`	Admin at location	Trainers and members awaiting approval.
POST	`/verifications/{userId}`	Admin at location	Approve or reject, reason required on rejection (`FR-AUTH-08`).
PUT	`/members/{id}/trainer`	Admin at location	Assign or reassign. Both parties notified.
DELETE	`/members/{id}/trainer`	Admin at location	Unassign.
GET	`/members/{id}/timeline`	Admin at location	Consolidated history (`FR-MEM-06`).
GET	`/members/{id}/health-screening`	Admin / assigned Trainer	**Sensitive.** Access audited (`NFR-PRV-02`).
GET	`/locations/{id}/trainers`	Any at location	Trainer roster.
GET	`/trainers/{id}`	Any at location	Profile, specialisations, certifications.
PUT	`/trainers/{id}/availability`	Self or Admin	Weekly availability windows.
GET	`/trainers/{id}/members`	Self or Admin	Assigned roster with adherence at a glance (`FR-TRN-05`).

> **There is no `POST /members`.** A member is not created by an administrator — a person registers, chooses a gym and location, and an admin then verifies them. Membership is a role grant, not an identity (LB-DATA-1.0 §2.1). The endpoint that admits someone is `POST /verifications/{userId}`, which is a decision about an existing person rather than the creation of one.

8. Billing — Plans, Memberships, Payments

Method	Path	Required	Purpose
GET	`/plans`	Any staff	Catalogue. Gym-wide.
POST	`/plans`	**Gym Owner**	Pricing is gym-wide; a Location Admin may sell from the catalogue but not alter it.
POST	`/members/{id}/memberships`	Admin at location	Sell a plan. Computes expiry.
POST	`/memberships/{id}/freeze`	Admin at location	Pause; extends expiry by the frozen days (`FR-BIL-08`).
POST	`/memberships/{id}/renewal`	Admin at location	New term from the later of today and current expiry.
POST	`/payments`	Admin at location	**Record money taken offline.** Idempotency required.
GET	`/payments`	Admin at location	Ledger. Filters: date range, method, collector, plan.
POST	`/payments/{id}/refund`	Admin at location	Never mutates the original (`FR-BIL-12`).
GET	`/payments/{id}/receipt`	Admin or the payer	GST receipt with a gapless number.
GET	`/locations/{id}/dues`	Admin at location	Outstanding balances (`FR-BIL-04`).
POST	`/members/{id}/pt-packages`	Admin at location	Sell session credits against a named trainer.
GET	`/me/billing`	Any	Own plan, expiry, dues, receipts (`FR-BIL-14`). Returns `null` membership for staff who train without a plan.

```
POST /api/v1/payments
Idempotency-Key: 7c2a…
{
  "memberId":    "01J9…M4",
  "amountMinor": 450000,              // ₹4,500.00
  "currency":    "INR",
  "method":      "UPI",
  "reference":   "UPI/429183746501",
  "receivedOn":  "2026-09-17",
  "allocations": [                     // one payment may settle several things
    { "targetType": "MEMBERSHIP", "targetId": "01J9…MB2", "amountMinor": 300000 },
    { "targetType": "PT_PACKAGE", "targetId": "01J9…PT9", "amountMinor": 150000 }
  ]
}
→ 201 Created
{ "paymentId": "01J9…PY3",
  "receipt": { "number": "IRON/2026-27/000318",   // gapless, per gym per FY
               "taxableMinor": 381356, "cgstMinor": 34322, "sgstMinor": 34322,
               "hsnSac": "999723" } }
```

> **Allocations are why partial payment works.** A payment points at what it settles through an allocation list rather than a single target field. Without it, a member paying ₹4,500 against a ₹3,000 membership and a ₹1,500 PT package would need two payment records for one transaction, and the ledger would no longer match the cash that actually changed hands.

9. Attendance

Method	Path	Required	Purpose
POST	`/locations/{id}/check-ins`	Self	**Check in.** QR or geofence. Idempotency required.
POST	`/locations/{id}/check-ins/manual`	Admin / Trainer	Mark on someone's behalf; acting user recorded.
POST	`/check-ins/{id}/check-out`	Self or staff	Optional, derives duration.
GET	`/me/check-ins`	Any	Own history and streak.
GET	`/locations/{id}/check-ins`	Admin at location	Footfall. Filters: date range, `purpose`, `flagged`.
GET	`/locations/{id}/check-ins/flagged`	Admin at location	Possible location spoofing, for review not rejection (`FR-ATT-11`).

```
POST /api/v1/locations/{locationId}/check-ins
Idempotency-Key: 9f1c8b30-6a4e-7c21-b8d5-1e2f3a4b5c6d
{
  "method":     "GEOFENCE",               // or "QR" with qrToken
  "latitude":   12.9352,
  "longitude":  77.6245,
  "accuracyM":  12,
  "purpose":    "PERSONAL_WORKOUT",       // SHIFT | PERSONAL_WORKOUT | BOTH
  "occurredAt": "2026-09-17T06:42:11+05:30",  // original time, preserved from the queue
  "deferred":   true                     // submitted late from the offline outbox
}
→ 201 Created
{ "checkInId": "01J9…CI8", "streakDays": 12, "distanceM": 34 }

// Replay of the same key — no second row, and the client can stop retrying
→ 200 OK   Idempotency-Replayed: true
```

> **`purpose` is not optional metadata.** When a trainer arrives they may be starting a shift, doing their own workout, or both. Without this field, trainer utilisation reports and member footfall inflate each other and every operational figure drifts (`FR-ATT-13` , `FR-ATT-14`). It is the small field that keeps the dashboard honest once staff start training on site.

10. Scheduling — Classes, Bookings, Personal Training

Method	Path	Required	Purpose
GET	`/class-types`	Any	Yoga, Zumba, meditation, HIIT. Gym-wide.
GET	`/locations/{id}/timetable`	Any at location	Recurring weekly rules.
POST	`/locations/{id}/timetable`	Admin at location	Add a recurring entry. Generates sessions forward.
GET	`/locations/{id}/sessions`	Any at location	Dated occurrences. Filters: `from`, `to`, `classTypeId`, `trainerId`, `deliveryMode`.
PATCH	`/sessions/{id}`	Admin at location	Amend one occurrence without touching the pattern (`FR-CLS-04`).
POST	`/sessions/{id}/cancellation`	Admin at location	Releases bookings, notifies, restores class-pack entries.
POST	`/sessions/{id}/bookings`	Self	**Book.** Waitlists automatically when full.
POST	`/bookings/{id}/cancellation`	Self	Late cancellation recorded if past cutoff.
GET	`/sessions/{id}/roster`	Session trainer / Admin	Who is booked.
POST	`/sessions/{id}/attendance`	Session trainer	Mark attended, absent or late (`FR-CLS-10`).
GET	`/sessions/{id}/meeting-link`	Booked member	**Revealed only within the window before start** (`FR-ONL-03`).
GET	`/me/bookings`	Any	Own schedule.
POST	`/pt-appointments`	Member or Trainer	Schedule within trainer availability.
POST	`/pt-appointments/{id}/completion`	Trainer	**Decrements a credit** and records session notes.
GET	`/me/pt-packages`	Any	Credit balance and expiry.

```
POST /api/v1/sessions/{sessionId}/bookings
Idempotency-Key: 4d8e…
→ 201 Created
{ "bookingId": "01J9…BK5", "state": "BOOKED", "waitlistPosition": null }

// Session full — this is a normal outcome, not an error
→ 201 Created
{ "bookingId": "01J9…BK6", "state": "WAITLISTED", "waitlistPosition": 3 }
```

> **A full class returns 201, not 409.** Being waitlisted is a successful booking in a different state — the member's request was understood and acted on. Returning an error would push the client into special-casing something the product treats as routine, and would lose the waitlist position in the process. `409 SESSION_FULL` is reserved for a genuine conflict: a session that permits no waitlist and has no places.

11. Training — Exercises, Workouts, Nutrition, Progress

> **Every endpoint in this section is `/me`-shaped or takes an explicit user id, and none takes a role.** A Gym Owner logging their own sets calls exactly the same endpoints a member does. That symmetry is `FR-PFP-01`, and it is the reason there is no `/members/{id}/workouts`.

Method	Path	Required	Purpose
GET	`/exercises`	Any	Catalogue. Filters: `muscleGroup`, `equipment`, `difficulty`, `discipline`, `q`. Cacheable with `ETag`.
GET	`/exercises/{id}`	Any	Instructions, muscle groups, demo media.
POST	`/exercises`	Admin / Trainer	Gym-specific addition (`FR-EXR-03`).
GET	`/muscle-groups`	Any	Global taxonomy.
GET	`/workout-templates`	Any staff	Reusable prescriptions.
POST	`/workout-templates`	Admin / Trainer	Exercises, sets, reps, load, rest.
POST	`/routines`	Admin / Trainer	Multi-week programme.
POST	`/users/{id}/routine-assignments`	Admin / Trainer	Assign to *any* account, staff included.
GET	`/me/today`	Any	**The training dashboard** (`FR-RPT-02`).
GET	`/me/workouts`	Any	Own history.
POST	`/me/workouts`	Any	**Log a workout.** Idempotency required — offline-capable.
GET	`/me/personal-records`	Any	Derived PRs.
GET	`/me/progress`	Any	Weight, measurements, body fat, trends.
POST	`/me/progress/metrics`	Any	Record a measurement.
GET	`/users/{id}/progress`	Assigned Trainer / Admin	Coaching view.
GET	`/me/diet-plan`	Any	Current plan and version history.
POST	`/users/{id}/diet-plans`	Admin / Trainer	Author after consultation (`FR-NUT-01`).
GET	`/me/diet-consultations`	Any	Consultation thread.

```
POST /api/v1/me/workouts
Idempotency-Key: 6b3f…
{
  "routineAssignmentId": "01J9…RA2",       // null for an unplanned workout
  "locationId":          "01J9…L1",
  "performedAt":         "2026-09-17T07:05:00+05:30",
  "durationMinutes":     52,
  "sets": [
    { "exerciseId": "01J9…EX1", "setNumber": 1, "reps": 8, "weightKg": 60.0, "rpe": 7.5 },
    { "exerciseId": "01J9…EX1", "setNumber": 2, "reps": 8, "weightKg": 62.5, "rpe": 8.0 }
  ]
}
→ 201 Created
{ "workoutLogId": "01J9…WL7",
  "newPersonalRecords": [
    { "exerciseId": "01J9…EX1", "recordType": "MAX_WEIGHT", "value": 62.5 }
  ] }
```

The whole workout posts as one request with its sets nested. Thirty sets as thirty requests would be thirty round trips on a gym-floor connection and thirty chances to half-succeed; as one body it is a single transaction that either records the session or does not.

12. Engagement

Method	Path	Required	Purpose
POST	`/feedback`	Any	Raise feedback with a category.
GET	`/locations/{id}/feedback`	Admin at location	Queue, filterable by state.
POST	`/feedback/{id}/responses`	Admin / Trainer	Threaded reply.
POST	`/users/{id}/coaching-notes`	Admin / Trainer	**Staff → member direction** (`FR-FBK-04`).
GET	`/me/inbox`	Any	Announcements, responses, coaching notes, notifications.
POST	`/announcements`	Admin	Broadcast to a location or the gym.
GET	`/me/notification-preferences`	Any	Categories, channels, quiet hours.
PUT	`/me/notification-preferences`	Any	Update.
POST	`/me/devices`	Any	Register an Expo push token.
DELETE	`/me/devices/{id}`	Any	Revoke on sign-out.
PUT	`/me/consents/{type}`	Any	Grant or withdraw. `MARKETING` and `WHATSAPP` are separate from `TERMS` (`NFR-PRV-06`).

13. Insights and Reporting

Method	Path	Required	Purpose
GET	`/dashboard`	Any	Role-appropriate dashboard. Always includes the training panel (`FR-RPT-01`).
GET	`/reports/revenue`	Admin	By period, location, plan, method.
GET	`/reports/attendance`	Admin	Footfall by day and hour. `purpose` separated, never merged.
GET	`/reports/class-performance`	Admin	Fill rate, waitlist depth, no-shows, ratings.
GET	`/reports/membership`	Admin	Joins, renewals, expiries, churn.
GET	`/reports/trainer-utilisation`	Admin	**Shift attendance only** (`FR-ATT-14`).
GET	`/reports/at-risk-members`	Admin	Churn worklist with contributing factors.
POST	`/reports/{type}/exports`	Admin	**Asynchronous.** Returns 202 with a job id.
GET	`/exports/{jobId}`	Requester	Poll; returns a signed download URL when ready.
POST	`/ai/generations`	Admin / Trainer	Request a draft. Never auto-published.
POST	`/ai/generations/{id}/approval`	Admin / Trainer	**Required before a member can see it** (`FR-AI-03`).

> **Exports are asynchronous by design.** A twelve-month revenue export across every location will not reliably finish inside an HTTP timeout, and `NFR-PRF-06` explicitly permits the escape. `202 Accepted` with a job id, a notification on completion, and a signed URL to collect — rather than a request that sometimes returns and sometimes times out depending on how much history a gym has accumulated.

14. Media

Method	Path	Purpose
POST	/media	Upload. `sensitivity` is a **required** field.
GET	/media/{id}/signed-url	Short-lived URL for a sensitive asset, issued after an authorisation check.
DELETE	/media/{id}	Remove. Refused while anything references it.

```
POST /api/v1/media        (multipart/form-data)
  file:        <binary>
  sensitivity: SENSITIVE          // REQUIRED — no default, ever
  purpose:     RECEIPT

→ 201 Created
{ "id": "01J9…MD3",
  "publicId": "lb/receipts/01J9MD3",   // NOT a URL
  "sensitivity": "SENSITIVE" }
```

The API never returns a media URL. It returns a `publicId`. Public assets have their URL constructed client-side with transformation parameters; sensitive assets require a call to `/signed-url`, which the server issues only after checking authorisation and which expires in minutes. There is no `url` field in any response and no `url` column in the database — which is what makes NN-4 unbreakable rather than merely documented. Progress photographs are out of scope entirely: no purpose value accepts them.

15. Public API

Unauthenticated, cacheable, and consumed by the Next.js public site at build and revalidation time. Everything here is deliberately shaped for a crawler and a CDN rather than for an application.

15.1 Reads

Method	Path	Returns
GET	`/public/gyms`	Published gyms for the directory and sitemap.
GET	`/public/gyms/{gymSlug}`	One gym's public profile.
GET	`/public/locations`	All published locations — drives the sitemap.
GET	`/public/gyms/{gymSlug}/{locationSlug}`	Address, geo, hours, phone — the `LocalBusiness` payload.
GET	`/public/gyms/{g}/{l}/timetable`	Upcoming public sessions — the `Event` payload.
GET	`/public/class-types`	Formats offered across the platform.
GET	`/public/locations/nearby`	Proximity search: `?lat=&lng=&radiusKm=`.

15.2 What the public API must never expose

> Member names or counts · any individual's attendance · payment or revenue figures · trainer contact details beyond what the gym published · anything belonging to a gym that is not `LIVE` and `isPubliclyListed` · internal identifiers that appear in the authenticated API.
>
> Public responses are built from a separate projection, not by stripping fields from an internal one. Stripping is a denylist, and a denylist fails open the day someone adds a field — which is how member lists end up in a search index.

15.3 The one public write

```
POST /api/v1/public/leads
Idempotency-Key: 8e2d…
{
  "gymSlug":      "iron-fitness",       // slugs, NOT ids
  "locationSlug": "koramangala",
  "name":         "Rahul Menon",
  "phone":        "+919812345678",
  "interest":     "TRIAL",
  "website":      ""                    // honeypot — must be empty
}
→ 202 Accepted
{ "received": true }
```

> **This endpoint is the platform's entire unauthenticated attack surface.** Four things make it defensible, and all four are required:
>
> • **Scope from slugs, never ids.** The server resolves `gymId` and `locationId` from the slugs. A body-supplied `gymId` would let anyone inject leads into any gym's pipeline.
>
> • **Rate limits** per IP and per phone (§2.8), with the IP stored hashed for abuse investigation rather than in the clear.
>
> • **A honeypot field** plus a minimum form-fill time. Both fail *silently* with 202 — telling a bot it was detected only helps it.
>
> • **RLS cannot protect this insert**, because there is no authenticated context to filter on. The service sets the scope explicitly, and that code path deserves a dedicated test.

16. Webhooks

Inbound only from providers we have integrated. LIMITBEYOND does not emit webhooks to third parties.

Path	Source	Handling
`/webhooks/twilio/status`	Twilio	OTP delivery status. Signature verified against the auth token; an unverified request is dropped without processing.
`/webhooks/whatsapp/status`	WhatsApp	Delivery and read receipts (`FR-WA-09`). Phase 4.
`/webhooks/expo/receipts`	Polled, not pushed	Expo receipts are fetched by a job rather than delivered — listed here so its absence is not mistaken for an omission.

Every webhook handler verifies the provider signature first, returns `200` quickly, and does its work through the job queue. A handler that processes inline will eventually time out and trigger the provider's retry, producing duplicate processing.

17. OpenAPI, Clients and Deprecation

17.1 Generation

springdoc generates OpenAPI 3.1 from the controllers at `/api/v1/openapi.json`, with Swagger UI at `/api/v1/docs` in non-production environments only. The document is committed on every change so a diff shows contract changes in review.

17.2 Clients

TypeScript types and a typed client are generated from that document into `packages/api-client` and consumed by both the mobile app and the web console. **Generated files are never hand-edited.** The benefit is that a breaking backend change fails the client build in CI rather than surfacing as a runtime error in a member's hands — which is most of what a shared-language backend would have given, recovered without abandoning Java (ADR-13).

17.3 Deprecation policy

Stage	What happens
Announce	Endpoint returns `Deprecation: true` and a `Sunset` date. Marked deprecated in OpenAPI, so the generated client warns at compile time.
Minimum notice	**90 days.** Long enough that a user who has not opened the app in two months still has a working build when they do.
Monitor	Usage tracked by client version. Removal happens when traffic reaches zero, not merely when the date passes.
Remove	In the next major version. Never within `v1`.

Additive changes are not deprecations and need no notice: new optional request fields, new response fields, new enum values, new endpoints. Clients must ignore unknown response fields and tolerate unknown enum values — a client that crashes on a new enum value is the reason APIs stop evolving.

18. Appendix A — Status Codes

Code	Meaning here	When
200	OK	Successful read, or an idempotent replay of a create.
201	Created	Resource created. `Location` header set. Includes a waitlisted booking.
202	Accepted	Work queued: exports, lead submission, OTP dispatch.
204	No Content	Successful delete or state removal.
304	Not Modified	Conditional request on cacheable reference data.
400	Bad Request	Malformed or failing field validation.
401	Unauthorized	Missing, invalid or expired token. Client should refresh once.
403	Forbidden	Authenticated, resource visible, action not permitted. **Not used for out-of-scope resources.**
404	Not Found	Absent **or outside the caller's scope** — deliberately indistinguishable (§1.2).
409	Conflict	State conflict: duplicate check-in, idempotency key reuse, last admin.
422	Unprocessable	Well-formed but rejected by a business rule: no credits, outside geofence.
429	Too Many Requests	Rate limited. `Retry-After` set.
500	Server Error	Unhandled. Correlation id returned; nothing internal disclosed.
503	Unavailable	A required dependency is down. `Retry-After` where known.

Note the 400 / 422 split: 400 means the request could not be understood as valid input, 422 means it was understood perfectly and refused on its merits. "No sessions remaining" is not a malformed request.

19. Appendix B — Error Code Catalogue

Code	Status	Meaning
VALIDATION_FAILED	400	Field-level validation. `fields` array present.
IDEMPOTENCY_KEY_REQUIRED	400	Missing on an endpoint that requires one.
IDEMPOTENCY_KEY_REUSED	409	Same key, different payload — a client bug.
UNAUTHENTICATED	401	No valid token.
TOKEN_EXPIRED	401	Distinct so the client refreshes rather than signing the user out.
FORBIDDEN	403	Visible resource, action not permitted.
ACCOUNT_PENDING	403	Awaiting admin verification.
ACCOUNT_SUSPENDED	403	Suspended or deactivated.
GYM_NOT_LIVE	403	Subscription lapsed past grace.
SCOPE_EXCEEDED	403	Attempt to grant beyond your own scope.
NOT_FOUND	404	Absent or out of scope.
DUPLICATE_CHECK_IN	409	Within the duplicate window.
LAST_ADMIN	409	Would leave a gym or location unadministered.
SESSION_FULL	409	No places and no waitlist permitted.
MEMBERSHIP_NOT_ACTIVE	422	Expired or frozen.
OUTSIDE_GEOFENCE	422	Beyond the location's radius.
NO_CREDITS_REMAINING	422	PT package exhausted.
CANCELLATION_CUTOFF_PASSED	422	Late cancellation may consume a credit.
AI_APPROVAL_REQUIRED	422	Attempt to publish unapproved AI content.
RATE_LIMITED	429	`Retry-After` set.

20. Appendix C — Worked Examples

A TRAINER WHO IS ALSO A LOCATION ADMIN OPENS THE APP

```
GET /api/v1/dashboard
→ 200
{
  "training": {                        // ALWAYS present, every role — FR-PFP-01
    "streakDays": 12,
    "todaysWorkout": { "name": "Push A", "exerciseCount": 6 },
    "nextSession":  { "className": "Power Yoga", "startsAt": "2026-09-18T06:30:00+05:30" },
    "membership":   null              // staff train without a plan — FR-BIL-16
  },
  "trainer": {
    "sessionsToday": 3, "assignedMembers": 24, "needingAttention": 5
  },
  "admin": {                           // ONLY for the location they administer
    "locationId": "01J9…L1",
    "activeMembers": 312, "pendingVerifications": 4,
    "checkInsToday": 87,
    "revenueThisMonth": { "amountMinor": 48750000, "currency": "INR" },
    "expiringIn30Days": 19
  }
}
```

Three panels, from one account, in one response. The `admin` panel covers `L1` only — this person trains at `L2` as well but administers nothing there, and the response reflects that without the client needing to reason about it.

A MEMBER TRIES TO REACH ANOTHER LOCATION'S ROSTER

```
GET /api/v1/locations/01J9…L9/members
→ 404 Not Found
{ "title": "Not found", "status": 404, "code": "NOT_FOUND",
  "detail": "That location was not found." }
```

The location exists. The response is indistinguishable from one for a location that does not, which is the point: an attacker enumerating identifiers learns nothing about which gyms are on the platform or how many locations they run.

> **Keeping this document honest.** The generated OpenAPI document is the contract; this is the reasoning around it. When an endpoint changes, change both in the same pull request — a specification that describes an API the code no longer implements is worse than none, because it is believed. The parts most worth re-reading before adding an endpoint are §1.2 (the three rules that outrank style) and §2.7 (idempotency), because both describe mistakes that are invisible in testing and expensive in production.

— END OF DOCUMENT —